SMART ANSWERING SYSTEM USING RETRIEVAL-AUGMENTED GENERATION (RAG)

PROJECT REPORT

Submitted by

GANESH A

Register No.: 711524MMC010

In partial fulfilment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

ANNA UNIVERSITY, CHENNAI - 600 025

Under the supervision and guidance

Of

Dr.E.VIJAYAKUMAR MCA.,Ph.D.,

Associate Professor &Head,

Department of Computer Applications

KIT-KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY

An Autonomous Institution

Accredited with 'A' Grade by NAAC NBA

COIMBATORE - 641 402

APRIL- 2026

KIT-KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY

An Autonomous Institution

Accredited with 'A' Grade by NAAC & NBA

COIMBATORE - 641 402

DEPARTMENT OF COMPUTER APPLICATIONS

M23CAP401 - PROJECT WORK

This is to certify that the project work entitled

SMART ANSWERING SYSTEM USING RETRIEVAL-AUGMENTED GENERATION (RAG)

Is the bonafide record of project work done by

GANESH A

Register No.: 711524MMC010

Of

MASTER OF COMPUTER APPLICATIONS

During the year

2025-2026

Project Guide **Head of the Department**

Submitted for the Project Viva-Voce Examination held on _____.

Internal Examiner External Examiner

DECLARATION

I affirm that the project work titled "**SMART ANSWERING SYSTEM USING RETRIEVAL - AUGMENTED GENERATION (RAG)**" is the original work of **Mr. GANESH A (Reg. No.: 711524MMC010)** being submitted in partial fulfilment for the award of the degree of **MASTER OF COMPUTER APPLICATIONS**. Certified further, that to the best of my knowledge the work reported here in does not form part of any other project report or dissertation work submitted for award of any degree or diploma, either in this or any other university on earlier occasion by me or any other candidates.

(Signature of the Candidate)
GANESH A
Reg. No.: 711524MMC010

We certify that the declaration made above by the candidate is true, to our knowledge.

Signature of the Guide
Dr.E.VIJAYAKUMAR MCA.,Ph.D.,
Associate Professor ,Head & Project Guide,
Department of Computer Applications

ACKNOWLEDGEMENT

The success of any project is the co-operative effort of the people around an individual. For all efforts, project, I am highly intended to the following personalities without whom this project would ever be completed.

At the outset, I would like to thank our Founder and Chairman **Thiru. PONGALUR N. PALANISAMY, KIT-Kalaignarkarunanidhi Institute of Technology**, who has given me an opportunity to undergo this Project Work, successfully in this esteemed Institution.

I express my sincere thanks to **Mrs. P. INDU MURUGESAN, Vice Chairperson, KIT-Kalaignarkarunanidhi Institute of Technology**, who encouraged me by giving her support and constant encouragement.

I extend my grateful thanks to **Dr. N. MOHANDAS GANDHIM.E., M.B.A., Ph.D., CEO, KIT-Kalaignarkarunanidhi Institute of Technology**, for the valuable suggestion in framing my carrier towards the fulfilment of this Project Work.

I express my sincere thanks to **Dr. K. RAMASAMY M.E., Ph.D., Principal,KIT- Kalaignarkarunanidhi Institute of Technology**, who encouraged me by giving his valuable suggestion and constant encouragement.

I would like to acknowledge the respective **Dr. E. VIJAYAKUMAR M.C.A., Ph.D., Associate Professor & Head, Department of Computer Applications, KIT-Kalaignarkarunanidhi Institute of Technology**, for spending the valuable time in guiding and supporting me to make this project a successful one.

I take the privilege to extend my thanks to guide **Dr. E. VIJAYAKUMAR M.C.A., Ph.D., Associate Professor & Head, Department of Computer Applications,** for spending their valuable time and energy in guiding, supporting and helping me in preparation of the project.

. Finally with great enthusiasm I express my thanks to all the faculty members for providing necessary information and their sustained interest in my part of successful completion.

ABSTRACT

The project “Smart Answering System Using Retrieval-Augmented Generation (RAG)” aims to develop an intelligent question-answering system that provides accurate, context- aware, and reliable responses by combining information retrieval with advanced language models. Unlike traditional chatbot systems that rely solely on pre-trained knowledge, the proposed system dynamically retrieves relevant information from a structured knowledge base or external documents and integrates it with a generative model to produce precise answers.The system utilizes embedding models to convert user queries and documents into vector representations, enabling efficient semantic search through vector databases such as ChromaDB . Upon receiving a query, the system retrieves the most relevant documents and feeds them into a large language model to generate contextually enriched responses. This approach ensures up-to-date, domain-specific, and explainable outputs.The proposed solution offers a scalable and cost-effective approach for applications such as customer support, academic assistance, and enterprise knowledge systems. It enhances answer accuracy, reduces hallucination, and improves user ineraction. Future enhancements include multimodal retrieval, real-time data integration, and improved reasoning capabilities through advanced LLM architecture.

CHAPTER NO	TITLE	PAGE NO
	LIST OF TABLES	
	LIST OF FIGURES	
1	**INTRODUCTION**	
	1.1 About the Project	1
2	**SYSTEM ANALYSIS**	
	2.1 Existing System	2
	2.1.1 Disadvantages	2
	2.2 Proposed System	3
	2.2.1 Advantages	4
	2.3 Feasibility Study	5
	2.3.1 Technical Feasibility	5
	2.3.2 Operational Feasibility	6
	2.3.3 Economic Feasibility	6
3	**SYSTEM SPECIFICATION**	
	3.1 Hardware Configuration	8
	3.2 Software Configuration	8
4	**SOFTWARE DESCRIPTION**	
	4.1 Introduction	10
	4.2 Frontend	10
	4.3 Backend	12
	4.3.1 Features	12
	4.4 Database Management System	14
	4.5 Frameworks and Libraries	14
5	**PROJECT DESCRIPTION**	
	5.1 Overview of the Project	15
	5.2 Modules Description	15

CHAPTER NO	TITLE	PAGE NO

	5.2.1 Login Page Module	16
	5.2.2 User Interface Module	16
	5.2.3 Query Input & Preprocessing Module	16
	5.2.4 Embedding Generation Module	17
	5.2.5 Information Retrieval Module	17
	5.2.6 Response Generation Module	17
	5.2.7 Database Module	17
	5.3 System Design	17
	5.3.1Input Design	18
	5.3.2Output Design	18
	5.3.3Data Flow Diagram	19
	5.3.4ER Diagram	22
	5.3.5Database Design	23
	5.4 Algorithm Used	26
	5.4.1 Training Process	26
6	**SYSTEM TESTING**	
	6.1 Unit Testing	28
	6.2 Integration Testing	29
	6.3 Functional Testing	30
	6.4 Performance Testing	30
	6.5 Validation Testing	31
7	**SYSTEM IMPLEMENTATION**	
	7.1 Implementation Overview	33
	7.2 Implementation Phases	33
	7.3 Implementation Challenges	34

CHAPTER NO	TITLE	PAGE NO
8	**CONCLUSION & FUTURE ENCHANCEMENT**	
	8.1 Conclusion	35
	8.2 Scope for Future Enhancement	36
9	**APPENDIX**	
	9.1 Source Code	37
	9.2 Screenshots	54
10	**REFERENCES**	
	10.1 Book References	57
	10.2 Web References	57
	10.3 Paper References	58

LIST OF TABLES

TABLE NO	TABLE NAME	PAGENO
5.3.1	User Query Table	23
5.3.2	Document Table	24
5.3.3	Embedding Table	24
5.3.4	Response Table	25
5.3.5	Login Table	25

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
5.3.1	Basic RAG system	20
5.3.2	RAG pipeline	21
5.3.3	Detailed System Process Flow	21
5.3.4	Entity-Relationship Diagram	22
9.2.1	Login Page	51
9.2.2	User Interface	52
9.2.3	User Uploading Processing	53
9.2.4	Answering Processing	54
9.2.5	Answering Query Generteing	55

CHAPTER 1
INTRODUCTION

1.1 ABOUT THE PROJECT

The increasing demand for intelligent and accurate information systems has led to the development of advanced question-answering solutions. Traditional chatbot systems rely mainly on pre-trained data, which often results in outdated, incomplete, or incorrect responses. These systems lack the ability to retrieve real-time or domain-specific information, making them less reliable for practical applications such as academic support, customer service, and enterprise knowledge management.To address these limitations, the proposed project, “Smart Answering System Using Retrieval-Augmented Generation (RAG),” combines information retrieval techniques with powerful language models to generate accurate and context-aware answers. Instead of depending only on stored knowledge, the system retrieves relevant information from external data sources such as documents, databases, or knowledge repositories and uses it to enhance response generation.The system works by converting user queries and stored documents into vector representations using embedding models. These vectors are stored in a vector database such as ChromaDB, enabling efficient semantic search. When a user submits a query, the system retrieves the most relevant information and provides it as context to a language model, which then generates a meaningful and accurate response.This approach improves answer quality, reduces incorrect or hallucinated responses, and ensures that the system remains scalable and adaptable to different domains. The proposed system can be used in various applications, including educational assistants, automated customer support, and knowledge-based systems.Overall, this project provides a cost-effective, efficient, and scalable solution for building intelligent answering systems, with future enhancements focusing on multimodal data handling, real-time information retrieval, and improved reasoning capabilities.

CHAPTER 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

The existing question-answering systems primarily rely on traditional chatbots or search- based approaches, which depend heavily on pre-trained language models or keyword-based search mechanisms and often fail to provide accurate, context-aware responses. Many chatbot systems generate answers solely based on their training data and lack access to updated or domain-specific information, resulting in outdated or incorrect outputs. Similarly, search engines return a list of documents or links rather than direct answers, requiring users to manually analyze and extract relevant information, which increases effort and reduces efficiency. Additionally, these traditional systems lack semantic understanding and struggle to interpret the true intent behind complex or ambiguous queries. Moreover, they do not effectively combine retrieval and generation techniques, leading to either unstructured information retrieval without proper summarization or generated responses without factual grounding, ultimately reducing reliability and usability in real-world applications such as education, customer support, and enterprise knowledge management.

2.1.1 DISADVANTAGE

The existing question-answering systems have several limitations:

- **Outdated Information:** Systems relying only on pre-trained models cannot access real- time or updated data, leading to inaccurate answers.
- **Keyword-Based Retrieval Issues:** Search-based systems depend on keyword matching, which may miss relevant information if the query wording differs.
- **No Direct Answers:** Search engines provide links instead of precise answers, increasing user effort and time.
- **Higher Chances of Hallucination:** Language models without

retrieval support may generate incorrect or misleading information.

- **Limited Domain Adaptability:** Existing systems are not easily customizable for specific domains such as healthcare, education, or business.
- **Scalability Challenges:** Managing large volumes of data and ensuring accurate retrieval becomes difficult without efficient vector-based systems

TOOLS BEING REPLACED OR ADDRESSED

- **Traditional Search Systems**: Conventional keyword-based search tools retrieve information based on exact word matching. However, they often fail to understand user intent and context, leading to irrelevant or incomplete results. These systems lack semantic understanding and do not generate intelligent responses.
- **Basic Chatbots**: Rule-based or scripted chatbots rely on predefined responses and decision trees. While they can handle simple queries, they struggle with complex, context-aware questions and cannot adapt dynamically to new information. Their responses are limited and lack depth.
- **Standalone Language Models (Without Retrieval)**: Large language models used without retrieval mechanisms generate responses based only on pre-trained knowledge. This can result in outdated or hallucinated answers, especially when handling domain-specific or real- time queries. They do not have access to external or updated data sources.
- **Manual Document Search and QA Systems**: Traditional document-based question answering requires users to manually search through files or databases. This process is time- consuming, inefficient, and prone to human error, particularly when dealing with large volumes of data

2.2 PROPOSED SYSTEM

The proposed "Smart Answering System Using Retrieval-Augmented Generation (RAG)" aims to provide an intelligent, accurate, and scalable solution for answering user queries by combining information retrieval with

advanced language models. Unlike traditional chatbot systems that rely only on pre-trained knowledge, this system dynamically retrieves relevant information from external sources such as documents, databases, or knowledge repositories to generate context- aware responses. It uses embedding models to convert user queries and documents into vector representations, enabling efficient semantic search through a vector database like ChromaDB or When a query is received, the system retrieves the most relevant information and passes it to a language model, which generates precise and meaningful answers based on the retrieved context. This approach improves answer accuracy, reduces hallucination, and enhances understanding of user intent. The system is fully automated, adaptable to different domains, and suitable for applications such as academic support, customer service, and enterprise knowledge systems,providing a cost-effective and efficient solution with potential for future enhancements like real-time data integration and multimodal capabilities.

2.2.1 ADVANTAGES

- **Improved Accuracy:**

  Retrieves relevant information from external sources, ensuring precise and reliable answers.

- **Reduced Hallucination:**

  Generates responses based on retrieved data, minimizing incorrect or misleading outputs.

- **Context-AwareResponses:**

  Understands user intent using semantic search and provides meaningful answers.

- **Real-TimeInformation Access:**

  Fetches up-to-date data from documents and databases instead of relying only on pre- trained knowledge.

- **Scalability:**

  Easily handles large datasets and multiple users using vector databases like ChromaDB or FAISS.

- **Domain Adaptability:**

  Can be customized for different fields such as education, customer support, and business applications.

- **Efficient Retrieval Mechanism:**

  Uses embeddings and similarity search for fast and accurate information retrieval.

- **Cost-Effective Solution:**

  Reduces dependency on manual search and improves efficiency with automated responses.

- **Future Expandability:**

  Supports enhancements like real-time data integration, multimodal inputs, and advanced reasoning.

2.3 FEASIBILITY STUDY

2.3.1 TECHNICAL FEASIBILITY

Data Availability

Assess the availability and quality of data sources such as documents, PDFs, databases, or knowledge repositories required for building the RAG system. Ensure that the data is relevant, well-structured, and accessible for embedding and retrieval processes. Ethical considerations such as data privacy, secure storage, and proper usage of information must be maintained to comply with applicable regulation

Model Complexity

Evaluate the feasibility of implementing embedding models and large language models for semantic search and response generation. Consider the computational requirements for generating embeddings, performing similarity search, and running inference on language models. Efficient model selection and optimization techniques are important to ensure fast and accurate responses.

Software and Hardware Requirements

Identify the required software tools such as Python, Flask (for API development), vector databases like ChromaDB or FAISS, and libraries such as Hugging Face Transformers for embedding and language models. Hardware

requirements include a system with a capable CPU, sufficient RAM, and optionally a GPU for faster model inference and processing large datasets.

Real-Time Operation:

The use of vector databases and efficient indexing techniques enables low-latency semantic search even with large datasets. The system is scalable to handle multiple users and large volumes of data. Future enhancements may include cloud deployment, real-time data updates, and integration with advanced models for improved performance and scalability.

Real-Time Performance and Scalability

The use of vector databases and efficient indexing techniques enables low-latency semantic search even with large datasets. The system is scalable to handle multiple users and large volumes of data. Future enhancements may include cloud deployment, real-time data updates, and integration with advanced models for improved performance and scalability.

2.3.2 OPERATIONAL FEASIBILITY

The Smart Answering System operates as a user-friendly web-based application that can be easily accessed through a browser without requiring complex setup or technical expertise. Once deployed, the system functions efficiently by processing user queries, retrieving relevant information, and generating responses in real time. The interface provides a simple and interactive dashboard for users to input queries and view results without needing command-line interaction. Additionally, the system ensures smooth operation with minimal user intervention, supports multiple users, and maintains consistent performance. Proper error handling and response management prevent system overload and ensure a reliable and seamless user experience.

2.3.3 ECONOMIC FEASIBILITY

The Smart Answering System is cost-effective as it primarily uses free and open-source technologies such as Python, Flask, Hugging Face Transformers, and vector databases like ChromaDB or FAISS. The system can be deployed on standard hardware with moderate specifications, eliminating the need for expensive infrastructure. Optional cloud services can be utilized based on requirements, but basic implementation does not require recurring subscription costs. Overall, the system minimizes development and operational expenses while delivering high performance, making it economically feasible for institutions and organizations.

CHAPTER 3
SYSTEM SPECIFICATION

3.1 HARDWARE CONFIGURATION

This section gives the details and specification of the hardware on which the system is expected to work.

- Processor : Intel Core i5 or higher.
- RAM : 12 GB of RAM or more.
- Hard Disk : 512 GB of disk space or more (SSD recommended).
- Keyboard : Standard keyboard.
- Monitor : 1280x720 resolution or higher recommended.

3.2 SOFTWARE CONFIGURATION

1. Operating System & Environment

- Ubuntu Linux (22.04 / 24.04 LTS) or Windows 10/11 for stable development and deployment.
- 64-bit (x86_64) architecture support for standard systems (optional ARM64 for edge devices).

2. Development & Frameworks

- Python 3.10+ for backend processing, embedding generation, and RAG pipeline.
- Flask / FastAPI for backend API services and request handling.
- HTML, CSS, JavaScript for front-end interface.
- Vector Databases (ChromaDB / FAISS) for semantic search and retrieval.

3. Libraries & Tools

- Hugging Face Transformers, Sentence Transformers for embeddings and language models.
- PyMuPDF, Python-docx, Pillow for document and image processing.
- OpenAI API for advanced language model integration.

4. Additional Tools

- VS Code / Jupyter Notebook for development.
- Git & GitHub for version control.
- Cloud Platforms for scalability and deployment.

CHAPTER 4

SOFTWARE DESCRIPTION

4.1 INTRODUCTION

This chapter explains the software technologies used to design and implement the Smart Answering System using Retrieval-Augmented Generation (RAG). The development of this project involves multiple software components, including programming languages, frameworks, vector databases, and development tools. Each component is carefully selected to meet the system's functional requirements such as query processing, semantic search, information retrieval, and response generation.The system follows a multi-tier architecture where user queries are handled through a web interface, data processing is performed by the backend server, and responses are generated using language models integrated with retrieval mechanisms. The backend manages embedding generation, vector database search, and response construction, while the frontend provides an interactive interface for users to input queries and view results.To implement this architecture, open-source technologies are widely used due to their flexibility, scalability, and strong community support. The combination of Python, Flask, Hugging Face Transformers, vector databases like ChromaDB or FAISS, and optional OpenAI APIs enables the system to process queries, retrieve relevant information, and generate accurate, context-aware responses efficiently through a user-friendly interface.

4.2 FRONT END

HTML, CSS, JAVA SCRIPT

The front-end of the Smart Answering System has been designed with a strong focus on user experience and simplicity. The interface allows users to easily input queries and receive accurate, well-structured responses in an interactive and user-friendly environment. The design ensures smooth navigation and efficient interaction, making the system suitable for students, professionals, and general users.HTML forms the backbone of the web

interface by providing the structure for input fields, response display areas, and chat-like interaction layouts. It ensures that user queries and system-generated answers are properly organized and presented clearly on the screen. CSS (Cascading Style Sheets) is used to enhance the visual appearance of the system by controlling layout design, colors, fonts, and responsiveness, ensuring compatibility across different devices and screen \sizes.JavaScript is used to make the interface dynamic and interactive. It enables real-time query submission and response display without reloading the page, providing a seamless chat experience. Additionally, JavaScript supports features such as loading indicators, chat history updates, and interactive UI components, improving overall usability and responsiveness of the system.

FEATURES

HTML:

HTML provides the basic structure of the web interface. It organizes elements like input fields, buttons, and response sections.

CSS:

CSS is used to control the presentation, formatting, and layout. It improves design using colors, fonts, spacing, and responsiveness.

JavaScript:

JavaScript controls the behavior of different elements on the webpage. It enables dynamic interaction like real-time query handling and response display.

USEAGE IN THIS PROJECT

- Provides accurate and context-aware answers to user queries
- Retrieves relevant information from documents and knowledge bases
- Generates meaningful responses using language models
- Allows users to interact through a simple web interface
- Supports academic learning by answering student questions
- Automates customer support by handling common queries
- Helps organizations manage and access internal knowledge

- Reduces manual effort and improves efficiency
- Delivers fast and real-time responses
- Ensures reliable and up-to-date information retrieval

4.3 BACKEND PYTHON

To understand how the back-end of the Smart Answering System works, it is important to see what happens when a user interacts with the application. When a user submits a query through the interface, the browser sends a request to the server hosting the system.

The Flask application on the server receives this request and processes it using the RAG pipeline. It converts the user query into embeddings, retrieves relevant information from the vector database, and passes the context to the language model to generate an accurate response. The final output is then sent back to the user's browser and displayed in real time.

Flask acts as a lightweight Python web framework that manages API routes and handles communication between the front-end and back-end. It connects different components such as the embedding model, vector database (ChromaDB), and language model, ensuring smooth data flow and efficient response generation. This enables the system to provide accurate, context-aware answers with a fast and seamless user experience.

4.3.1 FEATURES

Flask is widely popular because it offers the following advantages:

Efficient Query Processing:

Handles user requests and processes queries through the RAG pipeline quickly and efficiently.

Semantic Retrieval:

Retrieves relevant information from the vector database using embeddings for accurate context understanding.

Context-Aware Response Generation:

Integrates retrieved data with the language model to generate meaningful and accurate answers.

Modularity and Scalability:

The system is modular, allowing easy integration of new models, datasets, or features without major changes.

Real-Time Response Handling:

Ensures fast response generation and seamless communication between front-end and back-end.

USEAGE IN THIS PROJECT

- User enters a query through the web interface
- Browser sends request to the server
- Flask backend receives the request
- Processes the query using the RAG pipeline
- Converts user query into embeddings
- Retrieves relevant data from vector database (ChromaDB)
- Sends retrieved context to the language model
- Generates accurate and meaningful response
- Final response is sent back to the browser
- Output is displayed to the user in real time
- Flask manages API routes and communication
- Ensures smooth integration between all components

4.4 DATABASE MANAGEMENT SYSTEM

- Stores and manages system data efficiently
- Uses vector databases (ChromaDB / FAISS) for embeddings
- Enables fast semantic search and information retrieval
- Stores documents, queries, and response data

- Reduces data redundancy and improves organization
- Ensures quick access to relevant information
- Supports large-scale data handling and scalability
- Maintains data integrity, consistency, and security
- Integrates smoothly with backend components (RAG pipeline)
- Enhances overall system performance and accuracy

4.5 FRAMEWORKS AND LIBRARIES

- **Flask / FastAPI:** Used for backend development and API handling
- **Hugging Face Transformers:** Used for embedding models and language models
- **Sentence Transformers:** Converts text into vector embeddings
- **ChromaDB / FAISS:** Used as vector databases for semantic search
- **PyTorch / Torch:** Supports model execution and deep learning tasks
- **PyMuPDF & Python-docx:** Used for document parsing and extraction
- **Pillow (PIL):** Used for basic image processing (if needed)
- **OpenAI API :** Used for advanced response generation

CHAPTER 5
PROJECT DESCRIPTION

5.1 OVERVIEW OF THE PROJECT

The Smart Answering System using Retrieval-Augmented Generation (RAG) is an intelligent application designed to provide accurate, context-aware responses to user queries by combining information retrieval with advanced language models. It leverages modern AI techniques to retrieve relevant information from documents or knowledge bases and generate meaningful answers based on that context.Unlike traditional chatbots or search systems, which either rely on static knowledge or return only links, this system provides direct, precise, and up-to-date answers. It processes user queries in real time, retrieves the most relevant data using semantic search, and generates responses that are both informative and reliable.

By integrating embedding models, vector databases such as ChromaDB , and powerful language models, the system ensures efficient retrieval and high-quality response generation. The web- based interface allows users to interact easily, view responses instantly, and access information without manual searching.

Overall, the Smart Answering System offers a faster, more accurate, and scalable solution for question-answering compared to traditional methods, making it highly useful for applications such as education, customer support, and knowledge management systems

5.2 MODULE DESCRIPTION

The Smart Answering System is an intelligent, real-time question-answering application.

- Login page Module
- User Interface (Dashboard) Module
- Query Input & Preprocessing Module

- Embedding Generation Module
- Information Retrieval Module
- Responses Generation Module
- Database Module

5.2.1 Login page Module

The Login Page Module provides a secure and user-friendly entry point for users to access the system by entering valid credentials such as a username and password. It ensures that only authorized users can log in and use the application features. Upon successful authentication, users are redirected to the main dashboard where they can submit queries, view generated responses, and access their previous query history. The module is designed for simplicity and smooth navigation, and it may also include features such as input validation, error handling for incorrect credentials, password visibility options, and a “Forgot Password” feature to enhance both security and user experience.

5.2.2 User Interface (Dashboard) Module

Provides a centralized and user-friendly interface for users to interact with the system. It allows users to input queries, view generated responses, and access previous query history. The dashboard ensures smooth navigation and clear display of results, improving overall user experience. Additionally, it may include features such as search suggestions, response highlighting, and interactive elements for better usability.

5.2.3 Query Input & Preprocessing Module

Handles user queries entered through the interface. This module cleans, processes, and prepares the input text by removing unnecessary characters, handling special symbols, and normalizing the data. It ensures the query is structured properly for embedding generation, improving the accuracy of semantic search and response generation. Additionally, it may perform tokenization and basic validation to ensure meaningful input is passed to the next stage.

5.2.4 Embedding Generation Module

Converts user queries and stored documents into vector representations using embedding models such as Sentence Transformers. These embeddings capture the semantic meaning of the text rather than just keywords, enabling the system to understand context and intent. This module ensures that both queries and documents are represented in the same vector space, allowing efficient similarity comparison and accurate information retrieval in the next stage. Additionally, it supports faster processing and improves overall response quality.

5.2.5 Information Retrieval Module

Retrieves the most relevant information from the vector database (ChromaDB/FAISS) based on the similarity between query and document embeddings. It uses similarity search techniques such as cosine similarity to identify contextually relevant data. This module ensures that only the most meaningful and related information is selected, improving the accuracy and quality of the final response generated by the system

5.2.6 Responses Generation Module

Generates accurate and meaningful responses based on the retrieved information using a language model. It combines the user query with relevant context to produce clear, well-structured, and context-aware answers. This module ensures high-quality output by reducing irrelevant or incorrect responses and delivering precise information to the user in real time.

5.2.7 Database Module

Stores and manages documents, embeddings, and related data required for the system. It enables efficient storage and quick retrieval of information, supporting fast semantic search using vector databases like ChromaDB or FAISS. This module ensures organized data management, reduces redundancy, and maintains consistency, which improves the overall performance and accuracy of the system. Additionally, it supports scalability for handling large volumes of data.

5.3 SYSTEM DESIGN

5.3.1 INPUT DESIGN

Proper input design ensures that the system receives accurate and well-structured data for efficient processing. The Smart Answering System primarily accepts input in the form of user queries and optional document uploads, which are validated and prepared before processing.

➢ **User Query Input:**

Users enter queries through the interface in natural language. The system validates the input to ensure it is not empty or invalid and prepares it for further processing.

➢ **Document Upload Input:**

Users can upload documents such as PDFs, DOCX, or text files to expand the knowledge base. The system extracts and processes the content for embedding and storage.

➢ **Query Preprocessing:**

The input text is cleaned by removing unnecessary characters, handling special symbols, and normalizing the data. This ensures consistency and improves embedding accuracy.

➢ **System Validation:**

The system performs checks to ensure input quality and handles errors by providing appropriate messages for invalid or incomplete inputs.

5.3.2 OUTPUT DESIGN

` Output design defines how processed information is presented to users through the interface. The system ensures that responses are clear, accurate, and easy to understand.

➢ **User Dashboard:**

The interface displays user queries and generated responses in a structured format. It may include chat history and previously asked questions for better interaction.

➢ **Response Display:**

The system presents answers clearly with proper formatting, highlighting key information to improve readability.

➢ **Error Notifications:**
The system shows appropriate messages for invalid or empty queries to guide users effectively.

➢ **Real-Time Response:**
The backend processes queries and delivers responses instantly, ensuring a smooth and interactive user experience.

➢ **Contextual Output:**
Responses are generated based on retrieved data, ensuring they are relevant, accurate, and context- aware.

5.3.3 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is used to represent how data flows through the Smart Answering System and how different components interact with each other. It illustrates the movement of user queries, processing steps, and generated responses between system modules. The DFD helps in understanding how input data is transformed into meaningful output.

A Data Flow Diagram uses four basic symbols:

External Entity: External entities represent users or systems outside the application that interact with it. In this system, the primary external entity is the user who provides queries and receives responses.

Process: A process represents operations that transform input data into output data. In this system, processes include query preprocessing, embedding generation, information retrieval, and response generation.

Data Flow: Data flow represents the movement of data between entities, processes, and data stores. It is shown using arrows that indicate how queries, embeddings, retrieved data, and responses move through the system.

Data Store: A data store represents where data is stored within the system. In this project, it includes document storage and vector databases (ChromaDB/FAISS) that store embeddings and enable efficient retrieval of relevant information.

Notation	De Marco & Yourdon
External Entity	
Process	
Data Store	
Data Flow	

LEVEL 0

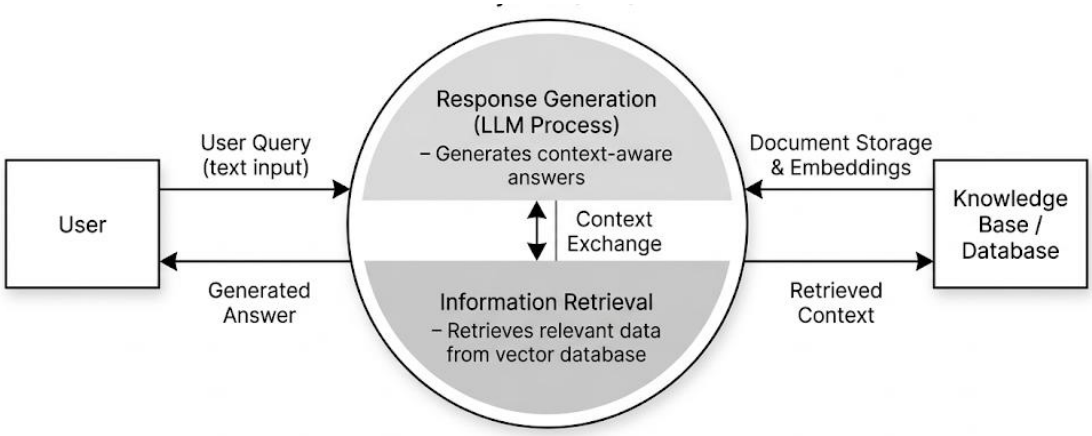

Figure 5.3.2: Basic RAG system

LEVEL 1

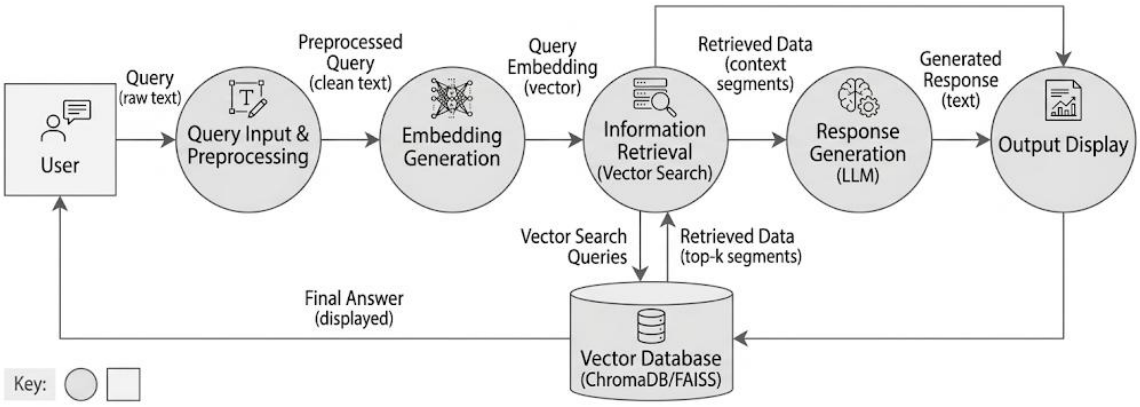

Figure 5.3.2: RAG pipleline

Level 2

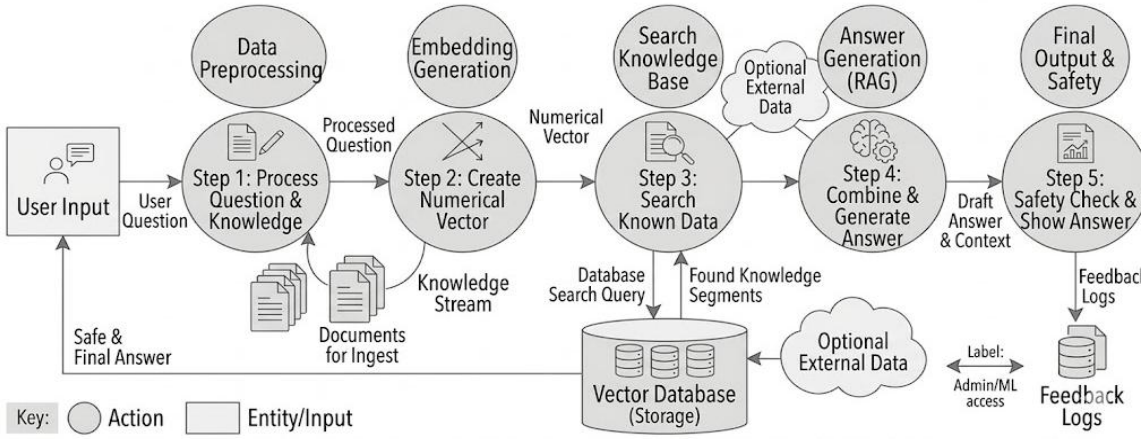

Figure 5.3. 3: Detailed System Process Flow

5.3.4 ENTITY-RELATIONSHIP (ER) DIAGRAM

The Entity-Relationship (ER) diagram represents the logical structure of the database used in the Smart Answering System using Retrieval-Augmented Generation (RAG). It illustrates the entities, their attributes, and the relationships among them.The system is designed to efficiently manage multiple components such as users, queries, documents, embeddings, and generated responses. These entities are interconnected to support smooth data flow, efficient storage, and accurate information retrieval within the system.

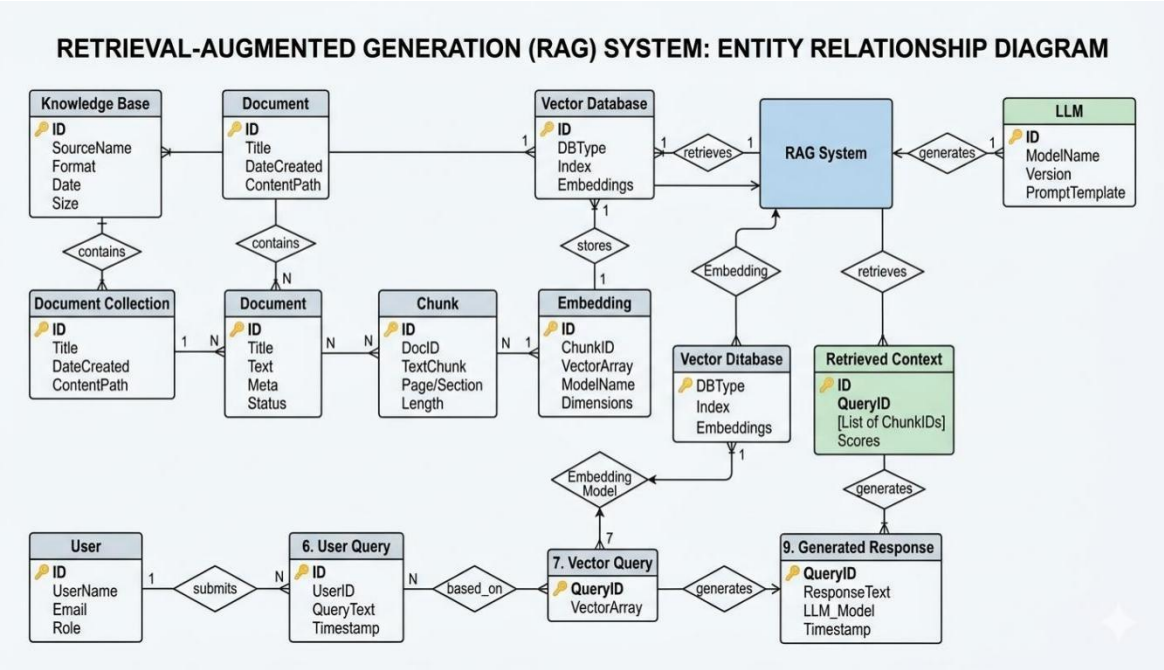

Fig. 5.3.4.1 Entity-Relationship (ER) Diagram

5.3.5 DATABASE DESIGN

Database design in the Smart Answering System using Retrieval-Augmented Generation (RAG) focuses on efficiently storing and retrieving data with minimal redundancy to support multiple applications. It is essential to develop a well-structured database that meets current information needs while remaining scalable for future enhancements. The design begins with a conceptual model that defines how data such as documents, embeddings, and metadata are organized and accessed within the system.

The key steps in developing the database design include:

- Data Analysis – Identifying and organizing the types of data such as user queries, documents, and embeddings.
- Relational Identity – Defining relationships between data components like queries, retrieved documents, and responses.
- Graphical Representation – Representing data structure and flow using diagrams for better understanding.
- Design Process – Structuring the database using vector storage (ChromaDB/FAISS) and efficient indexing techniques.

The database design plays a crucial role in system performance, testing, and maintenance. It reduces redundancy, improves retrieval speed, and ensures efficient data management. The primary objective is to enable fast and accurate response generation with minimal latency. The database in this system mainly includes three aspects: data storage (documents), vector storage (embeddings), and retrieval mechanism (semantic search).

DATABASE TABLE

1. User Query Table

Field Name	Data Type	Description
query_id	INT	Unique ID for each user

user_query	Text	Input query given by the user
timestamp	DateTime	Time of query submission

Table 5.3.1: User Query Table

2. Document Table

Field Name	Data Type	Description
doc_id	INT	Unique ID for each document
user_ document_text query	Text	Stored document content
source	Text	Source of the document

Table 5.3.2: Document Tab

3. Embedding Table

Field Name	Data Type	Description
embedding_id	INT	Unique ID for each document
doc_id	INT	Stored document content
vector	Vector	Numerical representation of text

Table 5.3.3: Embedding Table

4. Response Table

FieldName	Data Type	Description
response_id	INT	Unique ID for each document
query_id	INT	Stored document content
generated_response	Vector	Numerical representation of text
timestamp	DateTime	Time of response generation

Table 5.3.5.4: Response Table

5. Login table

Field Name	Data Type	Description
user_id	INT	Unique ID for each user
username	VARCHAR	Username entered by the user
password	VARCHAR	Encrypted password for authentication
email	VARCHAR	User email address for login and recovery
login_time	DATETIME	Timestamp of user login
status	VARCHAR	Login status (Success/Failed)

Table 5.3.5.5 login page Table

5.4 ALGORITHM USED

- **Retrieval-Augmented Generation (RAG):**

  Combines information retrieval with language models to generate accurate and context-aware

- **Embedding Algorithm (Sentence Transformers):**

Converts text into vector representations to understand semantic meaning

- **Similarity Measurement (Cosine Similarity):**

Calculates similarity between query and document embeddings to find relevant data

- **Vector Search (FAISS / ChromaDB):**

Efficiently retrieves nearest and most relevant documents from large datasets

- **Transformer-based Language Model (LLM):**

Generates meaningful and human-like responses based on retrieved context

Pseudocode

```
FUNCTION process_query(query)

  cleaned_query=preprocess(query)

  query_embedding = generate_embedding(cleaned_query)

  retrieved_docs = retrieve_from_database(query_embedding)

  context = combine_documents(retrieved_docs)

  response = generate_response(query, context)

  RETURN response

END FUNCTION
```

5.4.1TRAINING PROCESS

- **Data Collection:**

  Gather documents, PDFs, or text data to build the knowledge base

- **Data Preprocessing:**

  Clean and prepare text by removing noise and formatting content

- **Embedding Generation:**

  Convert documents into vector representations using embedding models

- **Vector Database Storage:**
  Store embeddings in vector databases like ChromaDB or FAISS
- **Query Embedding:**
  Convert user query into vector form during runtime
- **Similarity Search:**
  Retrieve relevant documents based on similarity (cosine similarity)
- **Context Formation:**
  Combine retrieved data to form meaningful context
- **Response Generation:**
  Use language model to generate final answer based on contex

CHAPTER 6
SYSTEM TESTING

After the development of the Smart Answering System using Retrieval-Augmented Generation (RAG), the system undergoes comprehensive testing to ensure accuracy, reliability, and efficient performance. The main objective is to verify that all components—such as query processing, embedding generation, information retrieval, and response generation—work correctly both individually and as an integrated system. Testing ensures that the system can handle different types of user queries, retrieve relevant information, and generate meaningful, context-aware responses in real time. Various testing methods, including unit testing, integration testing, functional testing, performance testing, and validation testing, are conducted to identify and eliminate errors, ensuring the system is stable, efficient, and ready for deployment

6.1 UNIT TESTING

Unit testing involves designing and executing test cases to verify that individual modules of the Smart Answering System function correctly and produce expected results. Each component such as query preprocessing, embedding generation, information retrieval, and response generation is tested independently before integrating into the complete system. This process ensures that the internal logic, data processing, and workflow of each module operate as intended. Unit testing verifies that user queries are properly processed, relevant information is accurately retrieved, and meaningful responses are generated, ensuring that each module meets the defined specifications and performs reliably.

Test ID	Component	Test Case	Result
UT-01	Query Module	Valid query input	PASS
UT-02	Query Module	Invalid/empty query	PASS
UT-03	Embedding Module	Embedding generation	PASS
UT-04	Database Module	Document storage & retrieval	PASS

UT-05	Retrieval Module	Relevant document retrieval	PASS
UT-06	Response Module	Accurate response generation	PASS
UT-07	API Module	End-to-end request handling	PASS

6.2 INTEGRATION TESTING

Integration testing focuses on verifying that the individual modules of the Smart Answering System work together seamlessly as a unified application. It ensures that data flows correctly between components such as query preprocessing, embedding generation, information retrieval, and response generation modules. Although each module functions correctly during unit testing, integration testing identifies and resolves issues that may arise during interaction, such as data mismatches, incorrect retrieval results, or response generation errors. This testing ensures smooth communication between all components, resulting in accurate, efficient, and consistent system performance.

Test ID	Modules Involved	Test Case	Result
IT-01	Query + Preprocessing	Query processed correctly	PASS
IT-02	Preprocessing+Embedding Module	Query converted to embeddings	PASS
IT-03	Embedding + Retrieval Module	Relevant documents retrieved	PASS
IT-04	Retrieval + Response Module	Context passed for response generation	PASS
IT-05	Response + UI Module	Response displayed correctly	PASS
IT-06	Database + Retrieval Module	Data fetched accurately database	PASS
IT-07	End-to-End Integration	Full query-to-response flow	PASS

6.3 FUNCTIONAL TESTING

Functional testing is conducted to ensure that the Smart Answering System performs all intended operations according to the project's requirements and specifications. It verifies that each core functionsuch as query processing, embedding generation, information retrieval, and response generation operates correctly and produces accurate results. Additionally, it ensures that internal processes like semantic search, context formation, and response generation work as expected. Overall, functional testing confirms that each module fulfills its purpose and that the system delivers reliable, consistent, and efficient responses to user queries

Test ID	Function Tested	Test Case	Result
FT-01	Query Input	User enters valid query	PASS
FT-02	Query Validation	System handles empty/invalid query	PASS
FT-03	Embedding Generation	Query converted into vector representation	PASS
FT-04	Information Retrieval	Relevant documents retrieved correctly	PASS
FT-05	Response Generation	Accurate response generated	PASS
FT-06	Result Display	Response displayed clearly on UI	PASS
FT-07	DocumentUpload (Optional)	File uploaded and processed successfully	PASS
FT-08	Error Handling	Proper error messages displayed	PASS

6.4 PERFORMANCE TESTING

Performance testing for the Smart Answering System using Retrieval-Augmented Generation (RAG) was conducted to evaluate how efficiently the system handles user queries in real time. The main objective was to ensure that the system processes queries quickly, retrieves relevant information accurately, and generates meaningful responses without delay. Tests were performed using different types of queries, including simple, complex, and domain-specific inputs, to measure response time, accuracy, and system resource usage. The system demonstrated high accuracy in retrieving relevant information and generating context-aware answers, while maintaining low latency and efficient performance. Even under multiple query

requests, the system remained stable and responsive. Overall, performance testing confirmed that the system is fast, reliable, and capable of delivering accurate results efficiently, making it suitable for real-time applications.

Test ID	Performance Aspect	Test Case	Result
PT-01	Response Time	System generates query response for simple inputs	PASS
PT-02	Complex Query Handling	Handles complex queries efficiently	PASS
PT-03	Load Testing	Multiple users simultaneously sending queries	PASS
PT-04	Retrieval Speed	Fast document retrieval from database	PASS
PT-05	Embedding Processing	Quick embedding generation	PASS
PT-06	System Stability	Stable performance under continuous usage	PASS
PT-07	Scalability	Handles increasing data volume efficiently	PASS

6.5 VALIDATION TESTING

Validation testing ensures that the Smart Answering System using Retrieval-Augmented Generation (RAG) meets all specified requirements and performs accurately in real-world scenarios. This testing focuses on verifying the accuracy, reliability, and usability of the integrated system, which includes modules such as query preprocessing, embedding generation, information retrieval, and response generation. The main objective is to confirm that the system retrieves relevant information correctly, generates accurate and context-aware responses, and presents them in a clear and user- friendly format.

During validation, the system is tested with a wide range of queries, including simple, complex, and domain-specific inputs. The relevance of retrieved information is evaluated against expected results, and the response generation is assessed to ensure correctness and clarity. The interaction between modules is also verified to confirm smooth data flow from user input to final output.

Additionally, the system's performance is tested to ensure quick response generation with minimal delay, even under multiple query requests.

Validation testing further examines usability by ensuring that users can easily interact with the interface, input queries efficiently, and understand the generated responses without confusion. Any issues such as irrelevant answers, incomplete responses, or retrieval mismatches are identified and improved through iterative testing.

Overall, validation testing confirms that the Smart Answering System satisfies both functional and non-functional requirements, delivering accurate, reliable, and efficient responses. This ensures that users can depend on the system for real-world applications while maintaining smooth performance and ease of use.

Test ID	Validation Aspect	Test Case	Result
VT-01	Response Accuracy	Correct answer generated for query	PASS
VT-02	Context Relevance	Retrieved data matches user query intent	PASS
VT-03	End-to-End Validation	Full system flow works correctly	PASS
VT-04	User Input Handling	Handles different types of queries	PASS
VT-05	Real-Time Response	Responses generated without noticeable delay	PASS
VT-06	System Reliability	Consistent results across multiple tests	PASS
VT-07	Usability Validation	Users can easily interact with the system	PASS

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1 IMPLEMENTATION OVERVIEW

The Smart Answering System using RAG is implemented as a web-based application using Python and Flask for backend and HTML, CSS, JavaScript for frontend. User queries are processed and converted into embeddings using embedding models.Relevant information is retrieved from a vector database like ChromaDB or FAISS.The retrieved data is passed to a language model to generate accurate responses.The system provides fast, reliable, and real-time answers with a user- friendly interface.

7.2 IMPLEMENTATION PHASES

The implementation of the Smart Answering System was carried out in multiple phases. In the first phase, User Interface Development, a simple and interactive frontend was designed to allow users to input queries and view responses. In the second phase, Data Preparation and Database Setup, documents were collected, processed, and stored in a vector database such as ChromaDB or FAISS for efficient retrieval.

The third phase focused on the Embedding and Retrieval System, where embedding models were used to convert text into vector representations and similarity search mechanisms were implemented to retrieve relevant information. In the fourth phase, Response Generation, a language model was integrated to generate accurate and context-aware answers based on retrieved data.

The final phase involved System Integration and Testing, where all modules were connected and tested to ensure smooth data flow, accuracy, and real-time performance. Additional features such as error handling, user-friendly interface improvements, and performance optimization were also implemented to enhance overall system efficiency.

7.3 IMPLEMENTATION CHALLENGES

During the development of the Smart Answering System, several technical challenges were encountered. One major issue was related to ensuring accurate information retrieval, where irrelevant or less meaningful documents were sometimes returned. This was addressed by improving embedding quality and fine-tuning similarity search techniques such as cosine similarity.

Another challenge involved handling noisy or unstructured data from uploaded documents, which affected embedding accuracy. This issue was resolved by implementing effective preprocessing techniques such as text cleaning and normalization. Additionally, managing response accuracy was challenging, as language models sometimes generated incomplete or less relevant answers. This was improved by providing better context from retrieved documents and refining prompt design.

System performance was also a concern when handling multiple queries or large datasets, leading to increased response time. This was optimized using efficient vector databases like ChromaDB/FAISS and improving query processing. Overall, these challenges were successfully addressed to ensure accurate, reliable, and efficient system performance.

CHAPTER 8
CONCLUSION& FUTURE ENCHANCEMENT

8. 1 CONCLUSION

The Smart Answering System using Retrieval-Augmented Generation (RAG) provides an intelligent and efficient solution for delivering accurate and context-aware responses to user queries. By combining information retrieval with advanced language models, the system is able to fetch relevant data and generate meaningful answers in real time, overcoming the limitations of traditional chatbots and search-based systems.

Unlike conventional methods that rely on static knowledge or keyword matching, this system dynamically retrieves information from external sources and produces precise, reliable, and up-to- date responses. Its modular architecture, including query processing, embedding generation, information retrieval, and response generation, ensures smooth operation and scalability across various applications such as education, customer support, and knowledge management.

Through extensive testing—including unit, integration, functional, performance, and validation testing—the system has demonstrated high accuracy, reliability, and efficiency. It provides a user- friendly interface and fast response time, making it suitable for real-world usage. Future enhancements such as real-time data integration, multimodal capabilities, and advanced reasoning can further improve its performance, making the Smart Answering System a robust, scalable, and effective solution for modern intelligent information systems.

8. 2 SCOPE FOR FUTURE ENHANCEMENT

In the future, the Smart Answering System using Retrieval-Augmented Generation (RAG) can be enhanced to provide more intelligent, accurate, and real-time responses. With continuous advancements in artificial intelligence, natural language processing, and data retrieval techniques, there is significant potential to improve the system's performance, scalability, and usability across various domains. Some key areas for future enhancement include:

- **Real-Time Data Integration:** Enable the system to retrieve and update information from live data sources, ensuring more up-to-date and dynamic responses.
- **Multimodal Capabilities:** Extend the system to handle multiple data formats such as images, PDFs, audio, and videos for richer and more interactive responses.
- **Advanced Reasoning and Context Understanding:** Incorporate more powerful language models to improve reasoning ability and handle complex, multi-step queries effectively.
- **Personalized Responses:** Introduce user-specific customization to provide more relevant and tailored answers based on user preferences and history.
- **Cloud-Based Deployment:** Implement cloud infrastructure to support scalability, remote access, and efficient handling of large datasets and multiple users.
- **Improved Retrieval Techniques:** Enhance vector search and ranking methods to improve the accuracy and relevance of retrieved information.
- **Interactive User Interface:** Develop more advanced dashboards with chat history, suggestions, and interactive features for better user experience.

CHAPTER 9
APPENDIX

9.1 SOURCE CODE

Index.Html

```
 <!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>Smart RAG Assistant</title>
 <link rel="stylesheet" href="/static/style.css" />
 <link rel="preconnect" href="https://fonts.googleapis.com" />
 <link
href="https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800&family
=DM+Sans:w ght@300;400;500&display=swap" rel="stylesheet" />
</head>
<body>
 <canvas id="bg-canvas"></canvas>
 <div class="overlay" id="overlay"></div>

 <div class="layout">
  <!-- SIDEBAR -->
  <nav class="sidebar" id="sidebar">
   <div class="sb-header">
    <div class="sb-logo">
     <div class="logo-icon">R</div>
     <span class="logo-text">Smart<span class="acc">RAG</span></span>
    </div>
    <div class="sb-header-actions">
     <button class="icon-btn" id="sidebarClose" title="Close sidebar">
      <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"><rect x="3" y="3" width="18" height="18" rx="2"/><path
d="M9 3v18"/></svg>
     </button>
     <button class="icon-btn" id="clearHistoryBtn" title="Clear history">
      <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"><path d="M3 6h18m-2 0v14a2 2  0 01-2 2H7a2 2 0 01-2-2V6m3
0V4a2 2 0 012-2h4a2 2 0 012 2v2m-6 5v6m4-6v6"/></svg>
     </button>
      <button class="icon-btn" id="newChatIcon" title="New chat">
```

```
        <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"><path d="M12 20h9"/><path d="M16.5 3.5a2.121 2.121 0 013
3L7 19l-4 1 1-4L16.5 3.5z"/></svg>
      </button>
    </div>
  </div>

  <div class="sb-section-label">Chats & Sessions</div>
  <div class="sb-scroll" id="historyContainer"></div>

  <div class="sb-footer">
    <div class="sb-stats-row" id="sbStats">
      <!-- populated by JS -->
    </div>
    <div class="user-row" id="userRow">
      <div class="user-avatar">U</div>
      <div class="user-info">
        <span class="user-name">User</span>
        <span class="user-sub">Free Plan</span>
      </div>
      <button class="icon-btn logout-btn" title="Sign out" id="logoutBtn">
        <svg width="14" height="14" viewBox="0 0 24 24" fill="none"
        stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M9
21H5a2 2 0 01-2-
2V5a2 2 0 012-2h4"/><polyline points="16 17 21 12 16 7"/><line x1="21"
y1="12" x2="9" y2="12"/></svg>
      </button>
    </div>
  </div>
</nav>

<!-- MAIN -->
<main class="main" id="main">
  <header class="topbar">
    <button class="icon-btn" id="sidebarOpen">
      <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke- width="2" stroke-linecap="round" stroke-
linejoin="round"><rect x="3" y="3" width="18" height="18" rx="2"/><path
d="M9 3v18"/></svg>
    </button>
    <div class="topbar-center">
      <span class="model-label" id="modelLabel">Smart RAG Assistant</span>
```

```
        <!-- Live accuracy badge -->
        <div class="acc-badge" id="accBadge" style="display:none">
          <span class="acc-dot"></span>
          <span id="accValue">—</span> accuracy
        </div>
      </div>
      <div class="topbar-right">
        <!-- Token usage chip -->
        <div class="token-chip" id="tokenChip" style="display:none">
          <svg width="12" height="12" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2"><circle cx="12"
cy="12" r="10"/><path d="M12 6v6l4 2"/></svg>
          <span id="tokenCount">0 tokens</span>
        </div>
        <button class="icon-btn" id="topNewChat" title="New chat">
          <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round"><path d="M12 20h9"/><path d="M16.5
3.5a2.121 2.121 0 013 3L7 19l-4 1 1-4L16.5 3.5z"/></svg>
        </button>
      </div>
    </header>

    <div class="thread" id="thread">
      <div class="welcome" id="welcome">
        <div class="welcome-orb"></div>
        <h1 class="welcome-heading">How can I help you today?</h1>
        <p class="welcome-sub">Ask questions about your uploaded knowledge
        base</p>
        <div class="starters" id="starters">
          <button class="starter-btn" data-q="Explain what this knowledge base
          contains">
            <span class="s-icon">      </span>
            <span class="s-title">Explore Knowledge Base</span>
            <span class="s-sub">Understand what's indexed</span>
          </button>
          <button class="starter-btn" data-q="Summarize the most important
documents in the knowledge base">
            <span class="s-icon">      </span>
            <span class="s-title">Summarize Key Documents</span>
            <span class="s-sub">Get a high-level overview</span>
```

```
      <!-- Upload Progress -->
      <div id="uploadProgressWrap" class="upload-progress-wrap"
      style="display:none;">
        <div id="uploadProgressBar" class="upload-progress-bar"></div>
      </div>
      <div id="uploadStatus" class="upload-status"></div>
      <div id="uploadMessage" class="upload-message" style="display:none;"></div>

  <p class="disclaimer">
    Smart RAG can make mistakes. Verify important information.
  </p>
  </div>

  <!-- Chunk Detail Modal -->

  <div class="modal-overlay" id="chunkModal" style="display:none">
    <div class="modal-card">
      <div class="modal-header">
        <span class="modal-title">Source Chunks</span>
        <button class="icon-btn" id="closeModal">✕</button>
      </div>
      <div class="modal-body" id="chunkBody"></div>
    </div>
  </div>

  <script src="/static/script.js"></script>
</body>
</html>
```

Style.css

```
/* ── RESET ── */ *, *::before, *::after { box-sizing: border-box; margin: 0;
padding: 0; }

:root { --sidebar-bg: rgba(8,12,20,0.92); --main-bg: transparent; --input-bg:
rgba(255,255,255,0.07); -
-hover-bg: rgba(255,255,255,0.07); --active-bg: rgba(0,229,160,0.1); --border:
rgba(255,255,255,0.08); --accent: #00e5a0; --accent2: #0088ff; --text-1: #f0f4ff; --
text-2: #8e9ab8; -- text-3: #4a5578; --send-active: #ffffff; --sidebar-w: 270px; --font:
'DM Sans', ui-sans-serif, system-ui, sans-serif; --font-display: 'Syne', sans-serif; --
radius: 14px; --bg-base: #080c14; }

html, body { height: 100%; overflow: hidden; }

body { font-family: var(--font); background: var(--bg-base); color: var(--text-1); font-
size: 15px; line- height: 1.6; -webkit-font-smoothing: antialiased; }
```

```
button { font-family: var(--font); cursor: pointer; border: none; background: none; }

::-webkit-scrollbar { width: 4px; } ::-webkit-scrollbar-track { background:
transparent; } ::-webkit- scrollbar-thumb { background: rgba(255,255,255,0.08);
border-radius: 99px; }

/* ── 3D BG CANVAS ── */ #bg-canvas { position: fixed; inset: 0; z-index: 0;
pointer-events: none; }

/* ── LAYOUT ── */ .layout { display: flex; height: 100vh; overflow: hidden;
position: relative; z- index: 1; }

/* ── OVERLAY ── */ .overlay { display: none; position: fixed; inset: 0;
background: rgba(0,0,0,0.6); z-index: 99; backdrop-filter:
blur(4px); } .overlay.show { display: block; }

/* ── SIDEBAR ── */ .sidebar { width: var(--sidebar-w); min-width: var(--
sidebar-w); background: var(--sidebar-bg); backdrop-filter: blur(24px); border-right:
1px solid var(--border); display: flex; flex-direction: column; flex-shrink: 0;
overflow: hidden; transition: width .25s cubic-bezier(.4,0,.2,1), min-width .25s
cubic-bezier(.4,0,.2,1); position: relative; z-index: 100; }

.sidebar.hidden { width: 0; min-width: 0; }

.sb-header { display: flex; align-items: center; justify-content: space-between;
padding: 14px 12px 10px; flex-shrink: 0; }

.sb-logo { display: flex; align-items: center; gap: 9px; }

.logo-icon { width: 30px; height: 30px; background: linear-gradient(135deg,
var(--accent), var(-- accent2)); border-radius: 8px; display: flex; align-items:
center; justify-content: center; font-family: var(--font-display); font-weight: 800;
font-size: 14px; color: #000; flex-shrink: 0; }

.logo-text { font-family: var(--font-display); font-weight: 700; font-size: 15px;
color: var(--text-1); letter-spacing: -.2px; white-space: nowrap; } .logo-text .acc
{ color: var(--accent); }

.sb-header-actions { display: flex; gap: 4px; }

/* Sidebar Upload */ .sb-upload { padding: 6px 12px 10px; flex-shrink: 0; border-
bottom: 1px solid var(--border); }

.sb-upload-btn { width: 100%; display: flex; align-items: center; justify-content:
center; gap: 8px; padding: 9px 14px; background: linear-gradient(135deg,
rgba(0,229,160,0.15), rgba(0,136,255,0.1)); border: 1px solid rgba(0,229,160,0.25);
border-radius: 10px; color: var(--accent); font-size: 13px; font-weight: 500;
transition: all .2s; } .sb-upload-btn:hover { background: linear-gradient(135deg,
```

rgba(0,229,160,0.22), rgba(0,136,255,0.16)); border-color: rgba(0,229,160,0.4); }

.upload-status { font-size: 12px; color: var(--text-3); margin-top: 6px; text-align: center; min-height: 16px; transition: color .2s; } .upload-status.success { color: var(--accent); } .upload-status.error
{ color: #ff5c5c; } .upload-status.processing { color: var(--accent2); }

.upload-message { font-size: 12px; color: var(--text-3); margin-top: 6px; text-align: center; min- height: 16px; }

.upload-progress-wrap { margin-top: 6px; height: 3px; background: rgba(255,255,255,0.08); border- radius: 99px; overflow: hidden; } .upload-progress-bar { height: 100%; width: 0%; background: linear-gradient(90deg, var(--accent), var(--accent2)); border-radius: 99px; transition: width .3s ease; }

.sb-section-label { font-size: 10px; font-weight: 600; text-transform: uppercase; letter-spacing: .08em; color: var(--text-3); padding: 14px 16px 6px; flex-shrink: 0; }

.sb-scroll { flex: 1; overflow-y: auto; overflow-x: hidden; padding: 4px 8px 8px; }

/* History items */ .hist-group-label { font-size: 10px; font-weight: 600; text-transform: uppercase; letter-spacing: .06em; color: var(--text-3); padding: 10px 8px 4px; white-space: nowrap; }

.history-item { padding: 8px 10px; border-radius: 8px; cursor: pointer; color: var(--text-2); font-size: 13px; white-space: nowrap; overflow: hidden; text-overflow: ellipsis; transition: background .15s, color .15s; user-select: none; position: relative; }

.history-item.chat-item { padding: 12px 16px; border-radius: 10px; cursor: pointer; transition: all 0.2s; margin: 2px 8px; }

.history-item.chat-item:hover { background: rgba(42,43,50,0.8); }

.history-item.chat-item.active { background: #343541; border-right: 3px solid #00e5a0; color:
white; } .history-item:hover { background: var(--hover-bg); color: var(--text-1); } .history-item.active
{ background: var(--active-bg); color: var(--text-1); border: 1px solid rgba(0,229,160,0.15); }

/* Per-session stats chip in sidebar */ .history-item .meta { display: none; font-size: 10px; color: var(-
-text-2); margin-top: 2px; } .history-item:hover .meta { display: block; }

/* Sidebar Footer */ .sb-footer { padding: 10px; border-top: 1px solid var(--border); flex-shrink: 0; }

```
.sb-stats-row { display: grid; grid-template-columns: 1fr 1fr; gap: 6px; margin-bottom:
8px; }

.stat-chip { background: rgba(255,255,255,0.04); border: 1px solid var(--border);
border-radius: 8px; padding: 7px 10px; text-align: center; } .stat-chip .sv { font-size:
15px; font-weight: 700; font-family: var(--font-display); color: var(--
accent); } .stat-chip .sl { font-size: 10px; color: var(--text-3); margin- top: 1px; }

.user-row { display: flex; align-items: center; gap: 10px; padding: 8px; border-radius:
8px; cursor: pointer; transition: background .15s; } .user-row:hover { background:
var(--hover-bg); } .user-avatar
{ width: 32px; height: 32px; border-radius: 50%; background: linear-
gradient(135deg, var(--accent), var(--accent2)); color: #000; display: flex; align-
items: center; justify-content: center; font-weight: 700; font-size: 13px; flex-shrink:
0; } .user-info { flex: 1; overflow: hidden; } .user-name { font-size: 13px; font-
weight: 500; display: block; white-space: nowrap; overflow: hidden; text-overflow:
ellipsis; } .user-sub { font-size: 11px; color: var(--text-3); display: block; }

.logout-btn { opacity: .5; } .logout-btn:hover { opacity: 1; }

/* ── ICON BUTTON ── */ .icon-btn { width: 32px; height: 32px; display: flex;
align-items: center; justify-content: center; border-radius: 7px; color: var(--text-2);
transition: background .15s, color .15s; flex-shrink: 0; } .icon-btn:hover
{ background: var(--hover-bg); color: var(--text-1); }

/* ── MAIN ── */ .main { flex: 1; display: flex; flex-direction: column;
overflow: hidden; min- width: 0; position: relative; }

/* ── TOPBAR ── */ .topbar { display: flex; align-items: center; justify-content:
space-between; padding: 8px 16px; flex-shrink: 0; gap: 8px; background:
rgba(8,12,20,0.5); backdrop-filter: blur(12px); border-bottom: 1px solid var(--
border); }

.topbar-center { position: absolute; left: 50%; transform: translateX(-50%);
display: flex; flex- direction: column; align-items: center; gap: 2px; pointer-
events: none; }

.model-label { font-family: var(--font-display); font-size: 14px; font-weight: 700;
color: var(--text-1); white-space: nowrap; }
```

```
.acc-badge { display: flex; align-items: center; gap: 5px; background:
rgba(0,229,160,0.1); border: 1px solid rgba(0,229,160,0.2); border-radius: 99px;
padding: 2px 9px; font-size: 11px; color: var(-- accent); font-weight: 500; white-
space: nowrap; } .acc-dot { width: 6px; height: 6px; border-radius: 50%;
background: var(--accent); animation: pulse 2s ease infinite; } @keyframes pulse
{ 0%,100%
{ opacity:1; transform:scale(1); } 50% { opacity:.5; transform:scale(.8); } }

.topbar-right { display: flex; align-items: center; gap: 8px; }

.token-chip { display: flex; align-items: center; gap: 5px; background:
rgba(255,255,255,0.05); border: 1px solid var(--border); border-radius: 99px; padding:
4px 10px; font-size: 11px; color: var(--text-2); }

/* ── THREAD ── */ .thread { flex: 1; overflow-y: auto; display: flex; flex-direction:
column; }

/* ── WELCOME ── */ .welcome { display: flex; flex-direction: column;
align-items: center; justify-content: center; flex: 1; padding: 40px 24px 0; gap:
0; animation: fadeIn .5s cubic- bezier(.16,1,.3,1) both; }

.welcome-orb { width: 72px; height: 72px; border-radius: 50%; background:
linear-gradient(135deg, var(--accent), var(--accent2)); margin-bottom: 22px;
position: relative; box-shadow: 0 0 40px rgba(0,229,160,0.3), 0 0 80px
rgba(0,136,255,0.15); animation: float 4s ease-in-out
infinite; } .welcome-orb::after { content: 'R'; position: absolute; inset: 0; display:
flex; align-items: center; justify-content: center; font-family: var(--font-display);
font-weight: 800; font-size: 28px; color: #000; }

@keyframes float { 0%,100% { transform: translateY(0); } 50% { transform:
translateY(-8px); } }

.welcome-heading { font-family: var(--font-display); font-size: clamp(22px, 4vw,
32px); font-weight: 800; color: var(--text-1); margin-bottom: 8px; text-align: center;
background: linear-gradient(135deg, var(--text-1) 60%, var(--accent)); background-
clip: text; -webkit-background-clip: text; -webkit-text- fill-color: transparent; }

.welcome-sub { font-size: 14px; color: var(--text-2); margin-bottom: 28px; text-align:
center; }

.starters { display: grid; grid-template-columns: repeat(2, 1fr); gap: 10px; max-width:
640px; width: 100%; }

.starter-btn { background: rgba(255,255,255,0.04); border: 1px solid var(--
border); border-radius: var(--radius); padding: 14px 16px; text-align: left;
display: flex; flex-direction: column; gap: 4px; cursor: pointer; transition:
```

```
background .2s, border-color .2s, transform .15s; color: var(--text-1); backdrop-
filter: blur(8px); } .starter-btn:hover { background: rgba(0,229,160,0.08);
border-color: rgba(0,229,160,0.25); transform: translateY(-2px); } .s-icon
{ font-size: 18px; margin-bottom:
2px; } .s-title { font-size: 13px; font-weight: 600; font-family: var(--font-
display); } .s-sub { font-size: 12px; color: var(--text-2); }

/* ── MESSAGES ── */ #messages { display: flex; flex-direction: column; padding-
bottom: 24px; }

.msg-row { width: 100%; padding: 8px 0; animation: fadeIn .3s cubic-
bezier(.16,1,.3,1) both; }

.msg-inner { max-width: 720px; margin: 0 auto; padding: 0 24px; display: flex; gap:
14px; align- items: flex-start; }

.msg-row.user .msg-inner { flex-direction: row-reverse; gap: 0; }

.msg-avatar { width: 28px; height: 28px; border-radius: 50%; display: flex;
align-items: center; justify-content: center; font-size: 11px; font-weight: 700;
flex-shrink: 0; margin-top: 2px; } .msg- avatar.ai { background: linear-
gradient(135deg, var(--accent), var(--accent2)); color: #000; } .msg- avatar.usr
{ background: rgba(255,255,255,0.15); color: var(--text-1); }

.msg-bubble { font-size: 15px; line-height: 1.7; color: var(--text-1); min-width: 0; flex:
1; }

.msg-row.user .msg-bubble { background: rgba(0,229,160,0.1); border: 1px solid
rgba(0,229,160,0.15); border-radius: 16px 16px 4px 16px; padding: 10px 16px; flex:
0 1 auto; max- width: 80%; margin-left: auto; }

.msg-meta { display: flex; align-items: center; gap: 8px; margin-top: 8px; flex-wrap:
wrap; padding- top: 8px; border-top: 1px solid var(--border); }

.meta-chip { display: inline-flex; align-items: center; gap: 4px; padding: 4px 8px;
border-radius: 6px; font-size: 11px; font-weight: 500; transition: all .15s; }

.meta-chip.chunks { background: rgba(0,136,255,0.12); border: 1px solid
rgba(0,136,255,0.3); color: #5ab4ff; cursor: pointer; }

.meta-chip.chunks:hover { background: rgba(0,136,255,0.2); }

.meta-chip.images { background: rgba(255,193,7,0.15); border: 1px solid
rgba(255,193,7,0.3); color: #ffc107; }

/* AI meta row */ .msg-meta { display: flex; align-items: center; gap: 10px;
margin-top: 8px; flex- wrap: wrap; }
```

Script.py

```
'use strict';

/* ── CONFIG ── */ // Dynamic base URL (works with any port:8000/8001/etc)
// Uses relative paths automatically

/* ── STATE ── */ let currentSession = { sessionId: null, filename: null }; let
currentChatId = null; let conversation = []; let chats = []; let isLoading = false; let
sidebarOpen = true;

/* ── HISTORY ── */ const historyContainer =
document.getElementById('historyContainer'); const clearHistoryBtn =
document.getElementById('clearHistoryBtn'); const modelLabel =
document.getElementById('modelLabel'); const sbStats =
document.getElementById('sbStats'); const tokenChip =
document.getElementById('tokenChip'); const accBadge =
document.getElementById('accBadge');

let allSessions = [];

/* ── DOM ── */ const sidebar = document.getElementById('sidebar'); const
overlay = document.getElementById('overlay'); const welcome =
document.getElementById('welcome'); const messages =
document.getElementById('messages'); const thread =
document.getElementById('thread'); const promptEl =
document.getElementById('prompt'); const sendBtn =
document.getElementById('sendBtn'); const fileInput
=document.getElementById('fileInput'); const uploadBtn =
document.getElementById('uploadBtn') || document.querySelector('.upload-icon');
const uploadStatus = document.getElementById('uploadStatus'); const
uploadMessage = document.getElementById('uploadMessage');

/* ── INIT ── */ bindEvents(); setSidebarState(window.innerWidth > 640);
loadChats(); loadSessions(); // Keep existing sessions sidebar option

/* ── NEW CHAT ── */ function newChat() { currentChatId = null;
console.log("New chat - Current Chat ID:", currentChatId); conversation = [];
messages.innerHTML = ''; promptEl.value = ''; if (currentSession.sessionId)
{ promptEl.disabled = false; promptEl.placeholder = `New chat about
${currentSession.filename} (has RAG context)`; modelLabel.textContent
= 'New Chat'; sendBtn.disabled = false; } else { welcome.style.display = 'flex';
promptEl.disabled = true; promptEl.placeholder = 'Upload a document to start
chatting...'; sendBtn.disabled = true; modelLabel.textContent = 'Smart RAG
Assistant'; } tokenChip.style.display = 'none'; loadChats(); }
```

```
/* ── RENDER MESSAGES ── */ function appendBubble(role, content)
{ appendBubbleWithMeta(role, content); }
```

App.py

```
import os import time import uvicorn
import traceback import uuid
from fastapi import FastAPI, File, UploadFile, HTTPException,
BackgroundTasks, Request from fastapi.middleware.cors import
CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import
FileResponse, JSONResponse from
pydantic import BaseModel
from werkzeug.utils import
secure_filename from openai
import OpenAI
from agent.planner import
agent_query from
ingestion.ingest_pdf
import ingest_pdf
from ingestion.ingest_docx
import ingest_docx from
ingestion.ingest_image import
ingest_image
from database.chroma_client import
delete_session_data from session_store
import session_store
from utils.token_counter import count_tokens_from_response
from utils.session_manager import add_to_history, get_chat_history,
get_session_stats from utils.chat_store import chat_store
from typing import Optional
from contextlib import asynccontextmanager

# ========================
# Globals
```

```
# ==========================
llm_client = None
OPENROUTER_API_KEY =
os.getenv("OPENROUTER_API_KEY")
UPLOAD_FOLDER = "data"

@asynccontextmanager
async def
    lifespan(app:
    FastAPI): #
    Startup
    global llm_client
    if not OPENROUTER_API_KEY:
        raise ValueError("[X] Please set OPENROUTER_API_KEY")

    llm_client = OpenAI(
        api_key="sk-or-v1-
        23818fbf7b2e55212884517dafadc6ac7c0ef1b5690986a8d4aa2091e6ef",

        base_url="https://openrouter.ai/api/v1"
    )
    os.makedirs(UPLOAD_FOLDE
    R, exist_ok=True) yield
#     FASTAPI
SETUP
# ==========================
app = FastAPI(lifespan=lifespan)

app.add_middle
    ware( CORS
    Middleware,
    allow_origins
    =["*"],
    allow_credent
    ials=True,
    allow_metho
    ds=["*"],
    allow_header
    s=["*"],
)

app.mount("/static", StaticFiles(directory="project_ui"),
name="static") app.mount("/data",
StaticFiles(directory="data", html=False), name="data")
```

```
# ========================
#     LLM FUNCTION - Smart RAG
Assistant (EXACT SPEC) #
========================
def ask_llm(text_context: str, images:
    list, question: str): """
    Smart RAG Assistant - follows exact
    rules from task. """image_keywords
    = ["show image", "diagram", "figure",
    "visual"] question_lower =
    question.lower()
    show_images = any(keyword in question_lower for keyword in image_keywords)
```

Main.py

```
import os from openai import OpenAI from agent.planner import agent_query
from database.chroma_client import get_text_collection from
ingestion.ingest_pdf import ingest_pdf from ingestion.ingest_image import
ingest_image

api_key = os.getenv("OPENROUTER_API_KEY") if not api_key: print("❌
ERROR: OPENROUTER_API_KEY not set.") print("1. Get free key:
https://openrouter.ai/keys") print("2. set OPENROUTER_API_KEY") print("3.
See .env.example") exit(1)

llm_client = OpenAI(
    api_key="sk-or-v1-
23818fbf7b2e55212884517dafadc6ac7c0ef1b56b74e90986a8d4aa2091e6ef",
base_url="https://openrouter.ai/api/v1" )

def ask_llm(context, question): prompt = f"""Use the following context to

answer the question. Context: {context}

Question: {question}"""

response =
    llm_client.chat.completions.creat
    e( model="meta-llama/llama-3.2-
    8b-instruct:free",
    messages=[{"role": "user",
    "content": prompt}]
)
```

```
return response.choices[0].message.content


if name == "main": print("    Multimodal RAG CLI")

# Check if vectors
already exist
collection =
get_text_collection()
if collection.count()
== 0:
    print("  Ingesting data...")
    ingest_pdf("data/Microsoft-Policymaker-Guide-
    Privacy.pdf")
    ingest_pdf("data/SMB_University_120307_Networking_Funda
    mentals.pdf") try:
        from ingestion.ingest_docx import
        ingest_docx
        ingest_docx("data/sample.docx")
    except ImportError:
        print("  python-docx not installed, skipping DOCX
(pip install python-docx)")


    except FileNotFoundError:
        print("  No sample.docx, skipping")
    ingest_image("data/Screenshot.png")  # Adjust if file
    exists print("✅ Ingestion complete")
else:
    print("✅ Using existing vectors")

while True:
    query =
    input("\nAsk
    (exit): ") if
    query.lower() ==
    "exit":
        break

    context =
    agent_query(query)
    answer =
    ask_llm(context,
    query)
    print("Answer:"
```

9.1 SCREENSHOTS

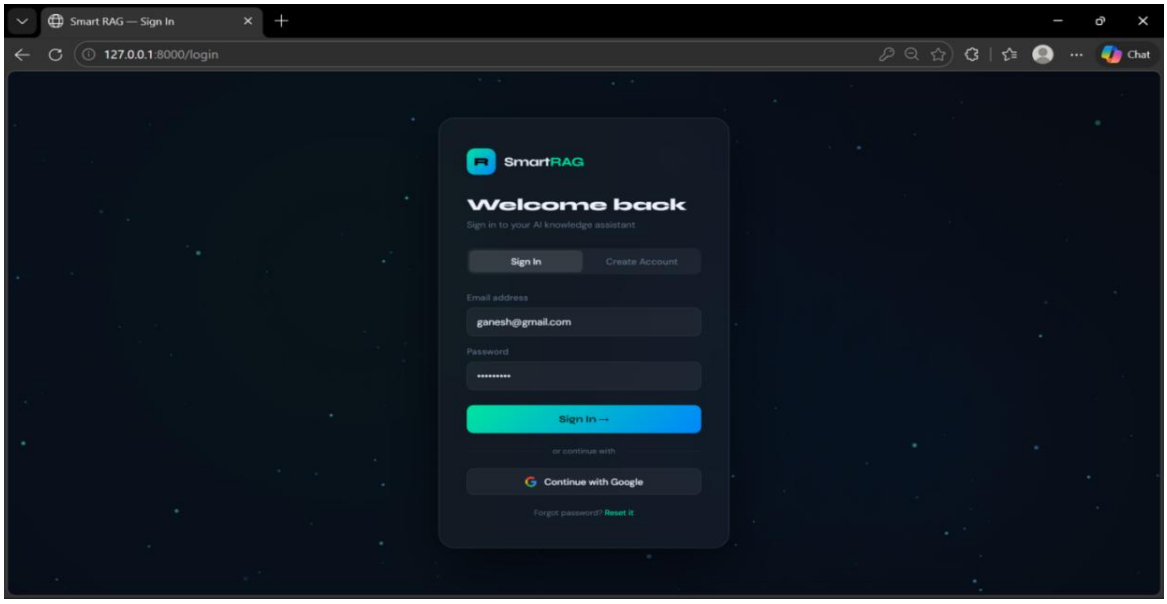

Figure 9.2.1 login page

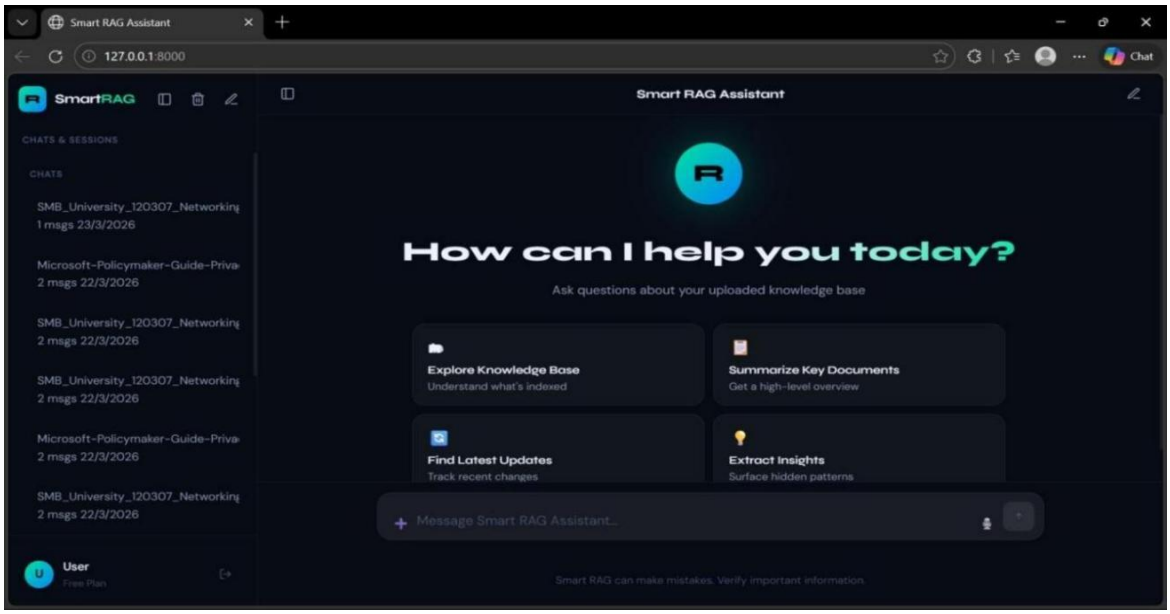

Figure 9.2.2 user_interface

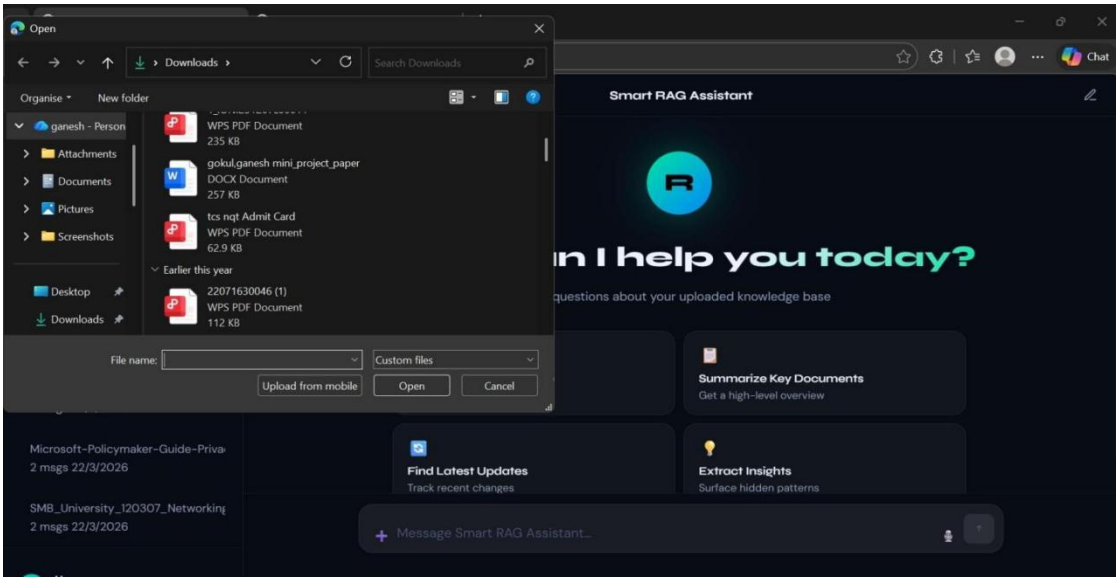

Figure 9.2.3 User uploading processing

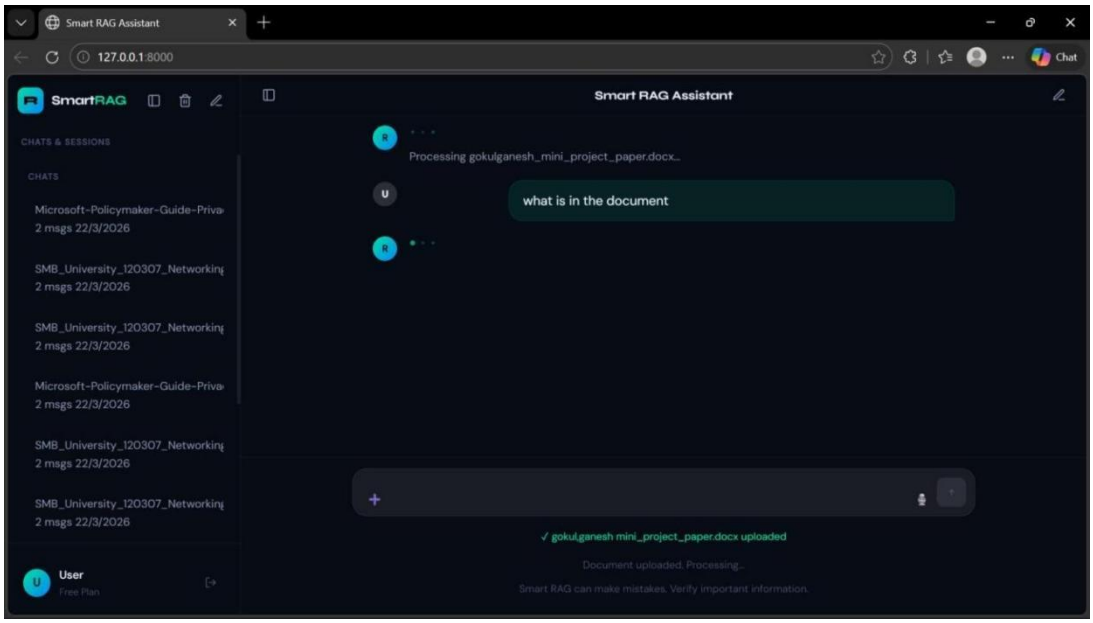

Figure 9.2.4 Answer processing

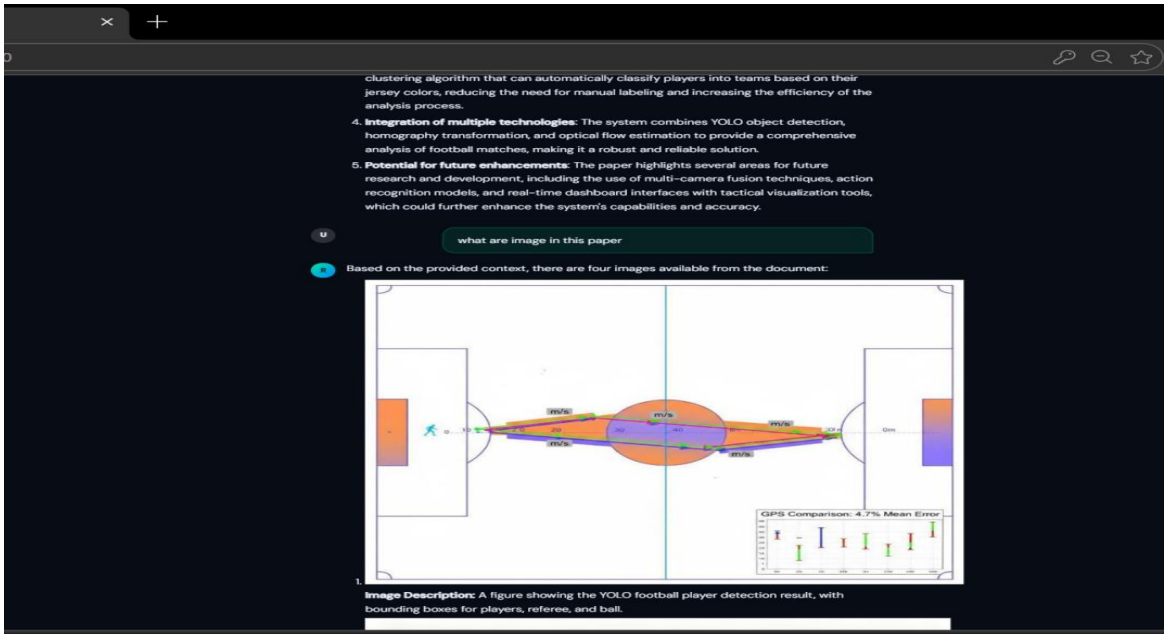

Figure 9.2.5 Answer generating

CHAPTER 10
REFERENCES

10.1 BOOK REFERENCES

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing*. Pearson.
- Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv preprint.
- Karpukhin, V., et al. (200). *Dense Passage Retrieval for Open-Domain Question Answering*. arXiv preprint.
- Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT- Networks*. arXiv preprint.
- Brown, T., et al. (2020). *Language Models are Few-Shot Learners*. arXiv preprint.
- OpenAI. (2023). *GPT Models and Applications*. OpenAI Documentation.
- Hugging Face. (2023). *Transformers Library Documentation*.

10.2 WEB REFERENCES

- OpenAI Documentation: https://platform.openai.com/docs/
- Hugging Face Transformers: https://huggingface.co/docs/transformers
- ChromaDB Documentation: https://docs.trychroma.com/
- FAISS Documentation (Facebook AI): https://faiss.ai/
- LangChain Documentation: https://python.langchain.com/docs/
- Sentence Transformers: https://www.sbert.net/
- Retrieval-Augmented Generation (RAG) Overview: https://arxiv.org/abs/2005.11401
- Dense Passage Retrieval (DPR): https://arxiv.org/abs/2004.04906

10.3 PAPER REFERENCES

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W. T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020).
  Retrieval-augmented generation for knowledge-intensive NLP tasks.
  arXiv preprint. https://arxiv.org/abs/2005.11401
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. T. (2020).
  Dense passage retrieval for open-domain question answering.
  arXiv preprint. https://arxiv.org/abs/2004.04906
- Reimers, N., & Gurevych, I. (2019).
  Sentence-BERT: Sentence embeddings using Siamese BERT-networks.
  arXiv preprint. https://arxiv.org/abs/1908.10084
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., et al. (2020).
  Language models are few-shot learners.
  arXiv preprint. https://arxiv.org/abs/2005.14165
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018).
  BERT: Pre-training of deep bidirectional transformers for language understanding.
  arXiv preprint. https://arxiv.org/abs/1810.04805